Measuring One Continuum Across a Clustered Sample Without Understating Its Uncertainty

A factor analytic workflow with replicate variance throughout, applied to institutional trust in Ecuador

Kevin Linares

2026-08-03

1 Purpose and scope

This is the factor-analytic arm of the workflow. It takes the same items, the same survey design, and the same replicate variance machinery as the latent class report, and asks a different question of them: not which of several kinds of respondent a person resembles, but where they sit on an underlying continuum.

The two reports share a configuration file and an engine. Only the measurement model and the scoring step differ, and the replicate design that produces every standard error is identical in both.

Almost nothing here is new statistics. Factor analysis of ordinal survey items is a mature method with mature software, and `lavaan` (Rosseel 2012) does the estimation. What the workflow adds is the design: the weights enter the estimation, and the standard errors come from the survey's own clustering rather than from an assumption that respondents were sampled independently. Section 7 shows what that is worth here, and the answer is not one anybody could have assumed in advance.

2 Choosing between a factor model and a latent class model

Both models take the same battery of answers and look for something underneath that the individual questions only measure imperfectly. They differ in what they assume that something is. A factor model assumes it is a single continuum running from low to high, and asks where on it each person sits. A latent class model assumes it is a small number of distinct kinds of respondent, and asks which kind each person most resembles. Neither is more correct in general; the battery decides.

The practical question is whether people differ in *how much* or in *which*. If respondents mostly line up from those who answer low on everything to those who answer high on everything, a continuum describes them well and any classes you extract are cut points on it. If instead a substantial group trusts some institutions while distrusting others, and another group reverses that pattern, no single number can hold both and the class model is recovering structure a factor cannot.

The factor model has two clear advantages when it applies. It estimates far fewer quantities, which matters a great deal at the sample sizes typical of a single-country survey: a five-class model on thirteen seven-category items carries several hundred free parameters, while a one-factor model on the same items carries under a hundred. And it treats the underlying trait as continuous, so it distinguishes between two people who both answer low without forcing them into the same bin.

Its disadvantage is communication. A class model reports a share of a group, and shares are what most readers already know how to read. A factor model reports a position on a scale with no natural units, which has to be translated before anyone outside the analysis can use it. Section 10 does that translation by converting positions back onto the original response scale, so a domain estimate reads as an expected answer rather than a standard deviation.

Both models rest on local independence: once you know a person's position on the factor, their answers to the individual questions carry no further relationship to each other. This is what lets either model be summarised so compactly, and it is the first thing to fail when a battery contains two questions that are really asking the same thing. Section 8 checks it and reports where it strains.

The assumption specific to the factor model is monotonicity: more of the trait always means a higher answer on every item. An item that violates this can still separate classes strongly while contributing almost nothing to a factor. Where the two arms disagree about an item, that disagreement describes the battery rather than indicating that one model is wrong.

3 Data and preparation

Preparation is identical to the class report. `survey_data_read.R` is shared between the two arms and holds the design columns and the demographic recodes; `survey_cfa_config.R` names the items for this one. Nonresponse codes become missing, items are recoded to consecutive integers on every row of the file, and demographic recodes are exhaustive with an error on any unmatched label.

The two arms carry separate item lists on purpose. A battery of one format asking one kind of question about different objects is what a factor model is built for, while a set of mixed-format items spanning unrelated domains is what a class model handles and a factor model cannot. Which battery suits which model is the substance of Section 2.

```r
dat <- cfg$data
w_vec <- dat[[cfg$weight]]
n_obs <- nrow(dat)
items <- cfg$items

design <- build_rep_design(dat, cfg)
des <- design$des
rep_des <- design$rep_des
nu <- degf(rep_des)
crit <- qt(0.975, nu)

# Effective sample size for the parallel analysis threshold. The Kish formula
```

```
# sees only the weights; the item-mean design effect sees the clustering too,
# which is what actually costs this survey information.
gdeff <- median(as.numeric(deff(svymean(reformulate(items), rep_des,
                                        deff = "replace", na.rm = TRUE))),
                na.rm = TRUE)
n_eff <- n_obs / gdeff

glue("{length(items)} items, {n_obs} cases. ",
     "Design: {ncol(rep_des$repweights)} replicates, {nu} degrees of freedom. ",
     "Weights: \nCV = {round(sd(w_vec) / mean(w_vec), 3)}, ",
     "unequal weighting effect = {round(1 + (sd(w_vec) / mean(w_vec))^2, 2)}.")
```

```
11 items, 1494 cases. Design: 84 replicates, 81 degrees of freedom. Weights:
CV = 0.054, unequal weighting effect = 1.
```

4 What the model says, and what lavaan is doing

The model says that each item is a noisy indicator of one underlying trait. Formally, for respondent i and item j,

$$x_{ij} = \tau_j + \lambda_j \eta_i + \varepsilon_{ij},$$

where η_i is respondent i's position on the factor, λ_j is the loading saying how strongly item j tracks it, τ_j is the item's intercept, and ε_{ij} is what the factor does not explain. A large loading means the item almost is the factor; a small one means it is measuring something else.

Our items are answered on a seven-point scale, not a continuous one, so `lavaan` does not fit the factor model above to the answers directly. It assumes an unobserved continuous response behind each item, cut into seven pieces by six thresholds, and estimates the thresholds along with the loadings. The correlations it works from are polychoric rather than ordinary correlations: they estimate the association between the two unobserved continuous responses rather than between the observed categories. The estimator is `WLSMV`, which is the standard choice for ordinal indicators and is what Mplus uses by default for the same problem.

Sampling weights enter through `sampling.weights`, so the estimated loadings and thresholds describe the population rather than the achieved sample. We use the marker method, fixing the first loading to one rather than standardising the factor. This is not cosmetic: it anchors the factor's *sign* as well as its scale, and Section 7 refits the model on every replicate, where `lavaan` starts cold each time. Without a fixed sign, a replicate could return a factor pointing the other way and the variance would absorb an enormous spurious deviation.

5 How many factors

Nothing here selects the number of factors. The report produces evidence and the analyst sets `cfg$n_factors` in the configuration, exactly as the class report does for the number of classes.

Three pieces of evidence, and they answer different questions.

Eigenvalues say how much shared variance each successive factor accounts for. The conventional check is Horn's parallel analysis: simulate data with no factor structure at all, compute its eigenvalues, and retain factors whose real eigenvalue clears the simulated one. Two adjustments matter here. The simulation runs at an effective sample size that divides the achieved one by the design effect of the item means, because a clustered survey of 1,600 carries less information than 1,600 independent observations and the threshold would otherwise sit too low. And the simulated data are cut into the same response categories as the real items and read back with the same polychoric estimator, because polychoric correlations are noisier than ordinary ones and a threshold built from continuous random data would be too easy to clear. The eigenvalues also carry confidence intervals from the replicate design, which a bare scree plot cannot offer.

Admissibility comes first, because it is a yes or no. A residual variance is the part of an item the factors do not explain, and it cannot be negative. When one comes out negative the solution is impossible rather than merely poor, and the usual cause is asking for more factors than the battery supports: one of them splits off and absorbs more than all of an item's variance. Such a solution is out regardless of how good its fit indices look, and the fit indices will look good, because an inadmissible solution is one that has fitted the sample too closely. Read `min_resid` beside the flag rather than trusting the flag alone: `lavaan` bounds residual variances at zero by default, so a Heywood case surfaces as a residual pinned near zero rather than as a negative one.

Fit indices say how well each candidate reproduces the observed correlations. They will improve as factors are added, always, because more factors fit better. They are useful for spotting a model that fits badly and close to useless for choosing between models that all fit acceptably.

Stability is what a search on an unknown battery most wants and is also the most expensive thing to compute, so the evidence here is indirect. The eigenvalues carry replicate-based confidence intervals, which is the check that costs nothing: a second eigenvalue whose interval clears the random-data threshold is not resting on a handful of sampling points, and one whose interval straddles it is not a factor whatever the fit indices say. Beyond that, the guard is the confirmatory fit itself. A structure read off this search and then fitted in Section 6 is not independently tested, which the limitations state, and on a battery where the eigenvalues do not settle the question the honest answer is a second sample rather than a resampling of this one.

```
R <- wcor(w_vec, items, dat)
eig_hat <- eigen(R, only.values = TRUE)$values

# Parallel analysis has to compare like with like. Polychoric correlations are
# noisier than Pearson ones, so a threshold built from continuous random data
# sits too low and the comparison leans toward retaining. Each draw is cut at
# the observed category proportions and read back with the same estimator.
cuts <- map(items, function(it)
  qnorm(head(cumsum(proportions(table(dat[[it]]))), -1)))
```

```r
set.seed(cfg$seed)
pa <- future_map(1:cfg$n_pa, function(i) {
  Z = matrix(rnorm(round(n_eff) * length(items)), round(n_eff))
  D = map2(seq_along(items), cuts,
           function(j, k) findInterval(Z[, j], k) + 1L) |>
    set_names(items) |>
    as_tibble()
  eigen(wcor(rep(1, nrow(D)), items, D), only.values = TRUE)$values
}, .options = furrr_options(seed = NULL,
     packages = c("lavaan", "dplyr", "purrr", "tibble"))) |>
  do.call(what = rbind)

eig_tbl <- tibble(factor = seq_along(items), eigenvalue = eig_hat,
                  se = sqrt(diag(replicate_variance(
                    rep_des,
                    function(w) eigen(wcor(w, items, dat),
                                      only.values = TRUE)$values,
                    eig_hat)))) |>
  mutate(lo = eigenvalue - crit * se, hi = eigenvalue + crit * se,
         random_95 = apply(pa, 2, quantile, 0.95),
         retain = lo > random_95)

eig_tbl |>
  filter(factor <= max(cfg$k_range)) |>
  ggplot(aes(factor, eigenvalue)) +
  geom_line(colour = viridisLite::viridis(1, begin = 0.4)) +
  geom_pointrange(aes(ymin = lo, ymax = hi), size = 0.3, na.rm = TRUE) +
  geom_line(aes(y = random_95), linetype = "dashed", colour = "grey45") +
  scale_x_continuous(breaks = cfg$k_range) +
  labs(x = "Number of factors", y = NULL) +
  theme_lca()
```

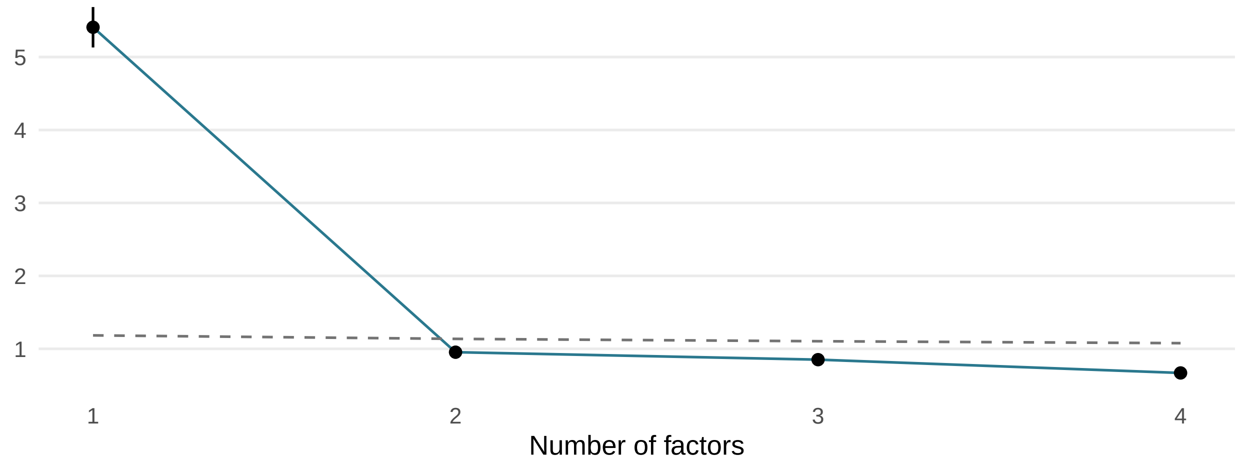


Figure 1: Design-weighted eigenvalues with replicate intervals against the random-data threshold.

Table 1: Design-weighted eigenvalues against a random-data threshold at the effective sample size.

factor	eigenvalue	se	lo	hi	random_95	retain
1	5.408	0.139	5.132	5.685	1.184	TRUE
2	0.954	0.044	0.866	1.042	1.137	FALSE
3	0.852	0.037	0.777	0.926	1.104	FALSE
4	0.669	0.026	0.618	0.721	1.078	FALSE

```r
efa_fits <- future_map(cfg$k_range, function(k) fit_efa(k, w_vec, items, dat),
                       .options = furrr_options(seed = NULL,
                         packages = c("lavaan", "dplyr")))

efa_tbl <- map2(cfg$k_range, efa_fits, function(k, f) {
  fit = as_fit(f)
  if(inherits(fit, "try-error")) return(tibble(k = k, converged = FALSE))
  fm = fitMeasures(fit)
  pick = function(nm) if(nm %in% names(fm)) as.numeric(fm[nm]) else NA_real_
  tibble(k = k, admissible = lavInspect(fit, "post.check"),
         min_resid = min(diag(lavInspect(fit, "theta"))),
         cfi = pick("cfi.scaled"), rmsea = pick("rmsea.scaled"),
         srmr = pick("srmr"))
}) |>
  list_rbind()

efa_tbl |>
  mutate(across(where(is.numeric), function(x) round(x, 3))) |>
  lca_table(align = "r",
            caption = "Factor solutions compared, admissibility first.")
```

Table 2: Factor solutions compared, admissibility first.

k	admissible	min_resid	cfi	rmsea	srmr
1	TRUE	0.397	0.946	0.108	0.048
2	TRUE	0.363	0.969	0.093	0.036
3	TRUE	0.336	0.988	0.068	0.021
4	FALSE	-0.862	0.996	0.045	0.013

5.1 Reading the loadings

A loading table is the factor-analytic equivalent of the class report's response profiles: it is where the analyst decides whether a solution makes sense. Loadings below 0.40 are blanked so the pattern is visible. Three flags are worth acting on. An item that loads nowhere is not being measured by any factor. An item that loads on two is ambiguous and usually means the solution has been pushed too far. An item whose communality falls below 0.30 shares almost nothing with the rest of the battery regardless of how many factors are extracted, and is a candidate for `cfa_drop`.

5.1.1 1 factors

Table 3: Standardised loadings at 1 factors, blanked below 0.40.

item	f1
respect_institutions	0.49
rights_protected	0.71
system_pride	0.72
system_support	0.65
armed_forces	0.52
legislature	0.70
police	0.61
political_parties	0.77
supreme_court	0.78
media	0.66
elections	0.71

Flagged: respect_institutions (low communality, h2 = 0.24); armed_forces (low communality, h2 = 0.27)

5.1.2 2 factors

Table 4: Standardised loadings at 2 factors, blanked below 0.40.

item	f1	f2
respect_institutions	0.49	.
rights_protected	0.68	.
system_pride	0.83	.
system_support	0.66	.
armed_forces	.	0.59
legislature	.	0.51
police	.	0.77
political_parties	.	0.53

Table 4: Standardised loadings at 2 factors, blanked below 0.40. *(continued)*

item	f1	f2
supreme_court	.	0.62
media	.	0.57
elections	.	0.47

Flagged: respect_institutions (low communality, h2 = 0.24)

5.1.3 3 factors

Table 5: Standardised loadings at 3 factors, blanked below 0.40.

item	f1	f2	f3
respect_institutions	0.50	.	.
rights_protected	0.53	.	.
system_pride	0.76	.	.
system_support	0.67	.	.
armed_forces	.	0.60	.
legislature	.	.	0.77
police	.	0.48	.
political_parties	.	.	0.85
supreme_court	.	.	0.67
media	.	.	.
elections	.	.	0.60

Flagged: respect_institutions (low communality, h2 = 0.25); media (loads nowhere, h2 = 0.25)

5.1.4 4 factors

Table 6: Standardised loadings at 4 factors, blanked below 0.40.

item	f1	f2	f3	f4
respect_institutions	0.53	.	.	.
rights_protected	0.53	.	.	.
system_pride	0.79	.	.	.
system_support	0.67	.	.	.
armed_forces	.	.	.	.
legislature	.	0.86	.	.
police	.	.	1.37	.
political_parties	.	0.86	.	.
supreme_court	.	0.63	.	.
media	.	.	.	1.01
elections	.	0.57	.	.

Flagged: respect_institutions (low communality, h2 = 0.28); armed_forces (loads nowhere, h2 = 0.13)

6 The chosen model

The eigenvalue evidence above favours a single factor, and two are fitted here. That is an analyst decision taken on the loading pattern rather than on the eigenvalues: the four system-support items and the nine institutional-trust items separate cleanly at two factors and load on constructs the questionnaire itself treats as distinct. The limitations record it as a departure from the automatic reading.

```
if (!k_ready)
  cat("**`cfg$n_factors` is `NULL`.** Read the evidence above, set the factor",
      "count and, if more than one, the item assignment in `cfg$cfa_factors`,",
      "then re-render. Everything below waits on that.\n")
```

```
if (!is.null(cfg$cfa_factors)) check_factors(cfg$cfa_factors, items)
stopifnot(is.null(cfg$cfa_factors) ||
          length(cfg$cfa_factors) == cfg$n_factors)
cat(cfa_syntax(items, cfg$cfa_factors, cfg$cfa_free), "\n")
```

```
f1 =~ respect_institutions + rights_protected + system_pride + system_support
  f2 =~ armed_forces + legislature + police + political_parties + supreme_court + media + elec
```

```
fit_w <- fit_cfa(w_vec, items, dat, cfg$cfa_factors, cfg$cfa_free)
fm <- fitMeasures(fit_w)

glue("Converged: {lavInspect(fit_w, 'converged')}. ",
     "{fm['npar']} free parameters on {n_obs} cases. ",
     "Robust CFI {round(fm['cfi.scaled'], 3)},
     TLI {round(fm['tli.scaled'], 3)}, ",
     "RMSEA {round(fm['rmsea.scaled'], 3)}, SRMR {round(fm['srmr'], 3)}.")
```

```
Converged: TRUE. 78 free parameters on 1494 cases. Robust CFI 0.97,
TLI 0.962, RMSEA 0.081, SRMR 0.038.
```

Conventional targets are CFI and TLI above 0.95, RMSEA below 0.06, and SRMR below 0.08. They are guidance rather than thresholds, they were derived under simple random sampling, and RMSEA in particular penalizes large samples. Read them alongside the loading pattern rather than instead of it.

7 Design-based variance

Every standard error reported here comes from the same stratified jackknife used in the class report: delete one primary sampling unit, reweight its stratum-mates, refit, and read the variance off the spread across refits. The estimator is stated in full in the class report; the point worth repeating is that the routine takes any function of a weight vector, so adding the factor model required no new variance machinery at all.

Two things are specific to this arm. **lavaan** starts cold on every replicate rather than warm-starting from the full-sample solution, so convergence is counted rather than assumed and a failed replicate is dropped with a note rather than treated as a zero deviation. And the comparison worth looking at is the last column: the ratio of the design-based standard error to the one **lavaan** reports by default, which assumes independent observations.

```r
par_names <- names(coef(fit_w))
Wm <- weights(rep_des, type = "analysis")

theta_cfa <- function(w_rep) {
  f <- try(fit_cfa(w_rep, items, dat, cfg$cfa_factors, cfg$cfa_free),
           silent = TRUE)
  if (inherits(f, "try-error") || !lavInspect(f, "converged"))
    return(rep(NA_real_, length(par_names)))
  coef(f)[par_names]
}

Theta <- do.call(rbind, future_map(seq_len(ncol(Wm)),
                                   function(r) theta_cfa(Wm[, r]),
                                   .options = furrr_options(seed = NULL)))
ok <- complete.cases(Theta)

d_dev <- sweep(Theta[ok, , drop = FALSE], 2, coef(fit_w)[par_names], "-")
V_cfa_par <- rep_des$scale * crossprod(d_dev * sqrt(rep_des$rscales[ok]))

param_tbl <- tibble(parameter = par_names,
                    estimate = as.numeric(coef(fit_w)[par_names]),
                    se_design = sqrt(diag(V_cfa_par)),
                    se_model = as.numeric(sqrt(diag(vcov(fit_w)))[par_names])) |>
  mutate(kind = case_when(str_detect(parameter, "=~") ~ "loading",
                          str_detect(parameter, "\\|") ~ "threshold",
                          .default = "variance"),
         deff = (se_design / se_model)^2,
         lo = estimate - crit * se_design, hi = estimate + crit * se_design)

glue("Replicates converged: {sum(ok)} of {nrow(Theta)}.")
```

```
Replicates converged: 84 of 84.
```

```r
param_tbl |>
  filter(kind == "loading") |>
  transmute(Parameter = str_remove(parameter, "^.*=~"),
            Estimate = sprintf("%.3f", estimate),
            `95% CI` = sprintf("[%.3f, %.3f]", lo, hi),
```

```
           `SE design` = sprintf("%.4f", se_design),
           `SE lavaan` = sprintf("%.4f", se_model),
           Deff = sprintf("%.2f", deff)) |>
  lca_table(align = "lrrrrr", widths = fit_widths(6, first = 1.8), font_size = 7,
           caption = "Loadings with design-based intervals, and the design
           effect on each.")
```

Table 7: Loadings with design-based intervals, and the design effect on each.

Parameter	Estimate	95% CI	SE design	SE lavaan	Deff
rights_protected	1.440	[1.273, 1.607]	0.0840	0.0574	2.14
system_pride	1.475	[1.307, 1.643]	0.0844	0.0589	2.05
system_support	1.318	[1.166, 1.470]	0.0764	0.0555	1.89
legislature	1.340	[1.207, 1.474]	0.0671	0.0493	1.85
police	1.172	[1.081, 1.262]	0.0455	0.0425	1.14
political_parties	1.465	[1.318, 1.613]	0.0741	0.0549	1.82
supreme_court	1.485	[1.341, 1.629]	0.0724	0.0548	1.74
media	1.262	[1.140, 1.384]	0.0612	0.0485	1.60
elections	1.355	[1.224, 1.486]	0.0660	0.0516	1.64

```
# The covariance between factors is estimated on the marker scale, which no one
# can read. The correlation is a function of three parameters the jackknife
# already carries, so its interval comes from the replicates rather than from a
# delta-method approximation.
fn_pair <- colnames(lavInspect(fit_w, "std")$lambda)

rho_of <- function(v, a, b)
  v[paste0(a, "~~", b)] / sqrt(v[paste0(a, "~~", a)] * v[paste0(b, "~~", b)])

cor_tbl <- if (length(fn_pair) > 1) {
  as_tibble(t(combn(fn_pair, 2)), .name_repair = function(x) c("a", "b")) |>
    pmap(function(a, b) {
      hat = as.numeric(rho_of(coef(fit_w), a, b))
      dev = apply(Theta[ok, , drop = FALSE], 1,
                  function(v) rho_of(v, a, b)) - hat
      se = sqrt(rep_des$scale * sum(dev^2 * rep_des$rscales[ok]))
      tibble(a = a, b = b, pair = paste(a, "and", b), rho = hat, se = se,
             lo = hat - crit * se, hi = hat + crit * se)
    }) |>
    list_rbind()
} else NULL

if (!is.null(cor_tbl))
```

```r
cor_tbl |>
  transmute(Pair = pair, Correlation = sprintf("%.3f", rho),
            `95% CI` = sprintf("[%.3f, %.3f]", lo, hi),
            `SE design` = sprintf("%.4f", se)) |>
  lca_table(align = "lrrr", font_size = 8,
            caption = "Correlation between factors, with a design-based interval.")
```

Table 8: Correlation between factors, with a design-based interval.

Pair	Correlation	95% CI	SE design
f1 and f2	0.843	[0.809, 0.876]	0.0169

The plot below is the diagram of the measurement model. It carries what a path diagram carries, one arrow per item, and adds what a path diagram cannot: the design-based interval on every loading. Loadings are on the marker scale rather than standardised, because that is the scale the jackknife estimated them on and rescaling would mean approximating the interval rather than reporting it. The anchor item is fixed at one by construction and has no interval.

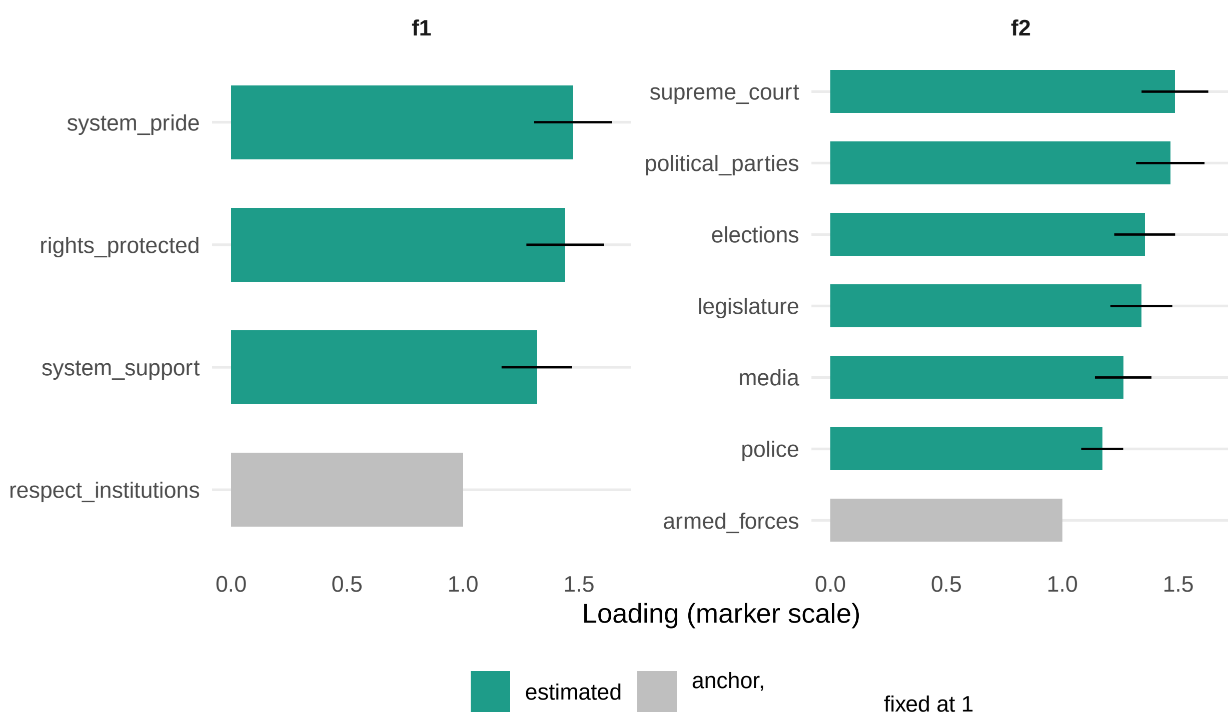


Figure 2: Loadings with 95 percent JKn intervals. The anchor item is fixed at 1 and has none.

The figure below is the fitted model drawn out: a node per factor, one label per item, and an edge whose weight is the standardised loading. It carries what a path diagram carries and is built from the same estimates the table above reports, rather than from a plotting package that would redraw lavaan's model-based numbers.

The number on each edge is the standardised loading, running from 0 to 1. It says how much of that item moves with the factor: near 1 and the item almost is the factor, near 0 and it is measuring something else. Squaring it gives the share of the item's variance the factor accounts for, so 0.7 means about half.

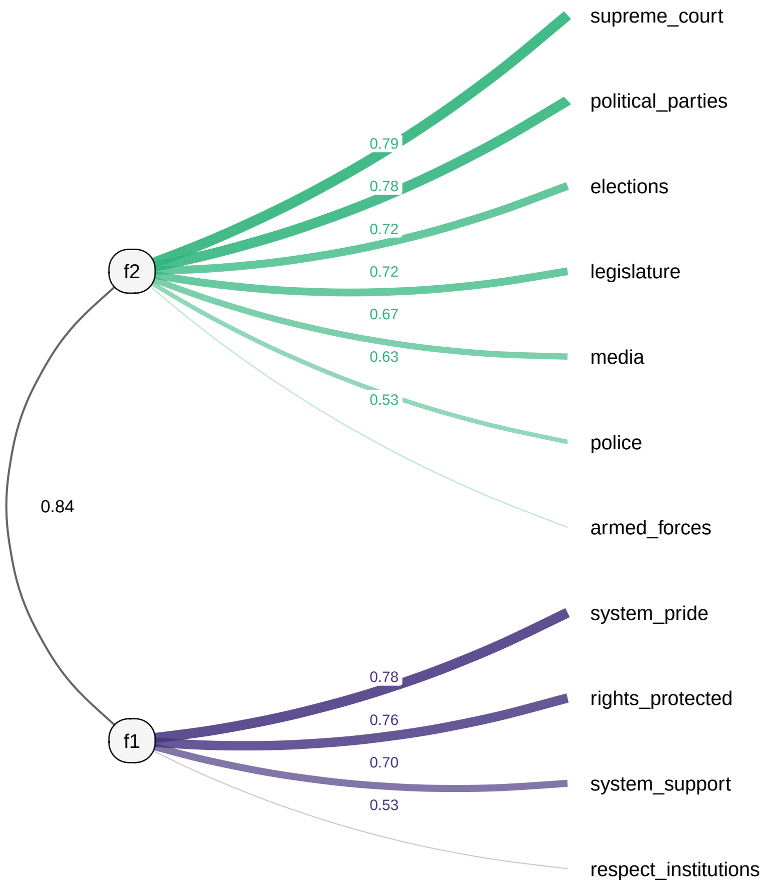


Figure 3: The fitted measurement model. Item edges carry the standardised loading; the arc between factors carries their correlation.

Table 9: Design effects by parameter type.

kind	n	median_deff
loading	9	1.82
threshold	66	1.07
variance	3	1.83

8 Fit diagnostics

Local independence is checked with modification indices. Each one asks how much the fit would improve if a particular constraint were relaxed, and the largest are almost always residual covariances between two items: a pair that shares something the factor does not capture.

Two cautions. Modification indices are computed from the model-based information matrix, which ignores clustering, so they inherit the same understatement the loading standard errors do and should be read as a ranking rather than a test. We do not quote the usual cutoff. And the expected parameter change matters more than the index: the index says the fit would improve, the change says by how much, and only the change is on a scale you can judge.

A pair that is both substantively coherent and large is a candidate for `cfg$cfa_free`. A pair that is large and arbitrary usually means an item should go.

Table 10: Ten largest modification indices for residual covariances. A ranking, not a test.

Item A	Item B	MI	EPC
armed_forces	police	85.9	0.191
political_parties	media	23.3	-0.113
legislature	media	22.9	-0.112
legislature	political_parties	20.0	0.085
armed_forces	political_parties	17.7	-0.111
media	elections	13.7	0.072
police	elections	12.8	-0.080
rights_protected	system_support	11.0	-0.075
armed_forces	elections	10.4	-0.078

9 Naming the factor

A factor is a number with no name. The model has told us which items track a continuum; it has not told us what that continuum is, and nothing in the estimation can. Naming it is an interpretive act, and this section drafts it with a language model under a fixed instrument so the analyst can accept, edit, or reject.

The design principle is the one that governs every language-model step in this workflow: the model receives conclusions, never parameters. It sees each item's wording and its standardised loading, ordered so the strongest indicators come first. It does not see the covariance matrix, the estimator, or the fit indices, because it cannot do anything useful with them and their presence degrades the output.

One instruction is specific to factor analysis and worth stating. A loading tells you an item belongs to the factor; it does not tell you which end of the scale is high. That comes from the response wording, so the response scale is supplied separately and the model is asked to state the high end explicitly as part of its answer. Without it a model will confidently name a factor backwards.

The prompt below is the real one, printed verbatim.

```
=========== SYSTEM PROMPT ===========
```

You are a senior survey methodologist who reads confirmatory factor analysis (CFA) measurement models. A factor is a single continuum running from low to high, and each item is a fallible measurement of it. An item's standardized loading says how strongly that item tracks the factor: near 1 means the item almost is the factor, near 0 means it carries something else. You name the factor from the items that load on it most strongly and the wording of those items, never from outside assumptions. A high loading tells you the item belongs, not which end of the scale is which; the item wording tells you that.

=========== USER PROMPT (f1) ===========

SURVEY CONTEXT
These items come from the 2023 AmericasBarometer survey of Ecuador, conducted by the LAPOP Lab at Vanderbilt University. Fieldwork was carried out face to face in Spanish by IPSOS between February and April 2023 with 1,604 respondents, drawn by a multi-stage probability design stratified by the three major regions of the country: Costa, Sierra, and Oriente.

All items analysed here are answered on the same seven-point ladder anchored at 1 (not at all) and 7 (a lot), shown to the respondent on a single card. Four ask how far the respondent supports the political system in general; the remaining nine ask how far the respondent trusts a specific national institution.
RESPONSE SCALE
All items are answered on a seven-point ladder where 1 means not at all and 7 means a lot.
ONE FACTOR FROM A CONFIRMATORY FACTOR ANALYSIS (CFA)
FACTOR f1, items ordered by loading:
system_pride loading 0.78 "Orgullo por el sistema político"
rights_protected loading 0.76 "Los derechos básicos están protegidos"
system_support loading 0.70 "Se debería apoyar al sistema político"
respect_institutions loading 0.53 "Respeto a las institutiones políticas"
armed_forces loading 0.00 "Confianza en las fuerzas armadas"
legislature loading 0.00 "Confianza en la legislatura"
police loading 0.00 "Confianza en la policía nacional"
political_parties loading 0.00 "Confianza en los partidos políticos"
supreme_court loading 0.00 "Confianza en el tribunal supremo"
media loading 0.00 "Confianza en los medios de comunicación"
elections loading 0.00 "Confianza en las elecciones"

TASK
Name what this factor measures, in two to five words, and describe it in one or two sentences. Lean on the items with the largest loadings. State which end of the scale is high using the response scale above, since the loadings do not tell you that.

RULES:
1. Use only the numbers and item wording shown. Survey context only
   clarifies what the items refer to; attribute nothing that the numbers

```
   do not show.
2. Quote a probability exactly as it is printed, for one response category
   at a time. Never add probabilities across categories and never describe a
   combined or total probability. If two adjacent answers both matter, name
   them separately with their own numbers.
3. Anchor every statement to the items that stand out most.
4. If nothing stands out, say the profile is diffuse rather than inventing
   a theme.
5. Return only valid JSON: no prose before or after, no markdown fences.
JSON (one object): {"label": "...", "description": "...", "high_end": "..."}
```

Names are frozen to `factor_labels.csv` in the output directory on first render and read from it thereafter, so a later render cannot quietly rename a factor. Editing that file is how the analyst takes over. Names never enter a computation: they are attached after every number is settled, so a wrong name is a presentation error rather than a statistical one.

```r
lab_path <- file.path(cfg$cfa_dir, "factor_labels.csv")

if (file.exists(lab_path)) {
  factor_labels <- read_csv(lab_path, show_col_types = FALSE)
} else {
  factor_labels <- map(fnames, function(fn) {
    obj <- parse_json_block(
      lca_chat(cfg, persona_cfa)$chat(
        prompt_factor_label(fit_w, dictionary, fn, scale_desc, cfg$survey_context),
        echo = FALSE))
    tibble(factor = fn,
           Label = pluck(obj, "label", .default = NA_character_),
           Description = pluck(obj, "description", .default = NA_character_),
           High_end = pluck(obj, "high_end", .default = NA_character_))
  }) |>
    list_rbind()
  write_csv(factor_labels, lab_path)
}

stopifnot(nrow(factor_labels) == length(fnames))

lca_table(factor_labels, widths = c("5em", "10em", "19em", "7em"),
          caption = "Factor names in use. Drafts: verify each against the
          loading table above before quoting it.")
```

Table 11: Factor names in use. Drafts: verify each against the loading table above before quoting it.

factor	Label	Description	High_end
f1	Political System Support	This factor is defined by pride in the political system, the belief that basic rights are protected, and general support for the political system.	7 (a lot)
f2	Trust in National Institutions	This factor is defined by trust in the supreme court (0.79), political parties (0.78), and elections (0.72). It measures the extent to which respondents trust the formal pillars of the state and political process.	7 (a lot)

10 Scoring respondents

10.0.1 Putting the factor on a scale a reader can use

A factor position is centred at zero with a standard deviation set by the model, and it is tempting to rescale it to run from 0 to 1 so it reads as a percentage. We do not, because the result would be a number that looks like a proportion and is not one: nothing in the model says a respondent at 0.6 has a 60 percent chance of anything, and a reader will assume otherwise.

Each respondent receives a position on the factor. Positions come out centred near zero on a scale with no natural units, which is unusable for anyone outside the analysis, so the domain tables carry a translation back to the response scale. The translation is a deliberate linearisation: take the observed mean of the factor's strongest indicator, and move it by the group's mean position times that item's standardised loading times the item's observed standard deviation. It uses the sample moments of the item rather than the model's thresholds, so it is an approximation that treats the seven-point ladder as if it were continuous, and it is reported because a group mean of 4.2 on a one to seven trust ladder is a statement anyone can check against the questionnaire. When an exact probability at a specific cut point is wanted, the fitted thresholds supply it directly from the same model, and that is the quantity to compute instead of rescaling a score.

Which kind of score matters, and the reason is the same one that motivates the bias correction in the class report. With ordinal indicators lavaan does not have the closed-form scores of the continuous case; the "regression" scores here are posterior modes, shrunk toward the grand mean in proportion to how reliably the factor is measured, and the "Bartlett" scores are per-pattern maximum likelihood positions computed by numerical optimisation, with no shrinkage toward the mean. The classical Bartlett result, that the score is conditionally unbiased for the factor and group means of it are therefore unbiased, is exact for continuous indicators; for ordinal indicators it holds approximately, and respondents with extreme response patterns can receive extreme positions since nothing pulls them in. The direction of the trade is unchanged: shrinkage pulls group means toward each other, so differences between demographic levels come out too small under the regression scores, exactly as classification error attenuates a cross-tabulation of modal assignments, while the maximum likelihood scores remove that attenuation at the cost of more error variance and wider intervals in Section 11. Wider and honest about group differences is the right side of that trade for a domain estimate.

The class report scores respondents who skipped some items, because local independence lets a missing answer drop out of the calculation. Under `WLSMV` that is not available, so the scored frame here equals the estimation frame. This is a real difference between the arms and it favours the class report where item nonresponse is substantial.

```r
# Two score types, for the same reason the class report shows two estimators.
# Regression scores are shrunk toward the grand mean by the factor's reliability,
# which pulls group means together; Bartlett scores remove that shrinkage at the
# cost of more error variance.
FS <- lavPredict(fit_w, method = "Bartlett")
FS_reg <- lavPredict(fit_w, method = "regression")
fnames <- colnames(FS)
scored <- bind_cols(dat, as_tibble(FS))
scored_reg <- bind_cols(dat, as_tibble(FS_reg))

design_s <- build_rep_design(scored, cfg)
rep_des_s <- design_s$rep_des
crit_s <- qt(0.975, degf(rep_des_s))

# A second design on the regression scores. Without it the design-based row and
# the Bartlett row read the same columns and come out identical, and the first
# gap would confound weighting with the choice of score.
design_reg <- build_rep_design(scored_reg, cfg)
rep_des_reg <- design_reg$rep_des

lam <- unclass(lavInspect(fit_w, "std")$lambda)
anchors <- map_chr(fnames, function(fn) rownames(lam)[which.max(abs(lam[, fn]))])
names(anchors) <- fnames

tibble(Factor = fnames, Label = factor_labels$Label,
       Mean = round(colMeans(FS), 3), SD = round(apply(FS, 2, sd), 3),
       Indicator = anchors,
       Loading = round(map_dbl(fnames, function(fn) lam[anchors[[fn]], fn]),
                       2)) |>
  lca_table(align = "llrrlr", widths = fit_widths(6, first = 1.6),
            font_size = 8,
            caption = glue("Factor positions across the {nrow(scored)} scored
                           respondents."))
```

Table 12: Factor positions across the 1494 scored respondents.

Factor	Label	Mean	SD	Indicator	Loading
f1	Political System Support	NA	NA	system_pride	0.78
f2	Trust in National Institutions	NA	NA	supreme_court	0.79

11 Domain estimation

The design is rebuilt on the scored frame and the mean factor position is estimated within each demographic level with `svyby`, so the intervals come from the same replicates as everything else.

11.0.1 What the design does to these intervals, and why the check matters either way

The tables below report each mean factor position with an unweighted and a design-based standard error side by side, and here the two sit close together: the design-based intervals are a median of five percent wider than the naive ones, reaching about twenty percent in the thinnest cells. The released weights are nearly constant, so weighting moves almost nothing, and a mean factor position within a demographic level averages over most of the eighty-four sampling points, which caps how much the clustering can inflate its variance. The summary line above the tables reports the median ratio exactly.

The point of computing the design-based interval is that nothing short of computing it tells you this. The naive interval answers a different question: what the uncertainty would be if these respondents had been drawn one at a time, independently, from the whole country. They were drawn in clusters of about nineteen, and people in the same neighbourhood answer alike. When an estimate concentrates within few clusters, or when the weights vary, the naive interval understates the uncertainty and its nominal 95 percent coverage fails, so an analyst using it calls differences significant that the data cannot support. The class report's battery and the production data with informative weights are exactly the settings where that gap opens. That the gap is small here is something the replicate design demonstrated rather than something anyone was entitled to assume.

Worth adding, because it stops the rule being mistaken for a bias toward caution: a design-based interval is not always wider. Stratification can make it narrower than the naive one, and a few cells below show precisely that. The rule is not "prefer the wider interval," it is "prefer the interval that matches how the sample was actually drawn."

Two limits belong with the numbers. Factor scores are estimated rather than observed, so differences between demographic levels are understated for the same reason a class model's cross-tabulations are: a difference that appears is real, a null is not evidence of absence. And a level resting on very few respondents will produce a wide interval that should not be read as a finding; sample sizes are reported beside every estimate for that reason.

```r
marginals <- map(cfg$aux, function(v) {
  m <- svymean(reformulate(v), rep_des_s, na.rm = TRUE)
  raw <- scored |> filter(!is.na(.data[[v]])) |>
    count(.data[[v]], name = "n") |> set_names(c("level", "n"))
  tibble(variable = v, level = str_remove(names(coef(m)), paste0("^", v)),
         weighted = as.numeric(coef(m)), se = as.numeric(SE(m))) |>
    left_join(mutate(raw, level = as.character(level)), by = "level")
}) |>
  list_rbind()

# Three estimators, matching the class report. Unweighted is a plain group mean
```

```r
# with an iid standard error. Design-based adds the weights and takes its
# variance from the replicates. Bartlett additionally removes the shrinkage that
# regression scores impose on group differences.
dom_one <- function(frame, des, cr, est) {
  crossing(variable = cfg$aux, factor = fnames) |>
    pmap(function(variable, factor) {
      if (is.null(des)) {
        d = frame |>
          filter(!is.na(.data[[variable]])) |>
          group_by(level = as.character(.data[[variable]])) |>
          summarise(p = mean(.data[[factor]]),
                    se = sd(.data[[factor]]) / sqrt(n()), .groups = "drop")
      } else {
        sb = as.data.frame(svyby(reformulate(factor), reformulate(variable),
                                 des, svymean, na.rm = TRUE))
        d = tibble(level = as.character(sb[[1]]), p = sb[[2]], se = sb[[3]])
      }
      a = anchors[[factor]]
      mutate(d, variable = variable, factor = factor,
             segment = match(factor, fnames), estimator = est,
             expected = mean(as.numeric(dat[[a]]), na.rm = TRUE) +
               p * lam[a, factor] * sd(as.numeric(dat[[a]]), na.rm = TRUE),
             lo = p - cr * se, hi = p + cr * se)
    }) |>
    list_rbind()
}

est_levels <- c("Unweighted", "Design-based", "Bartlett")

domain <- bind_rows(
  dom_one(scored_reg, NULL, qnorm(0.975), "Unweighted"),
  dom_one(scored_reg, rep_des_reg, crit_s, "Design-based"),
  dom_one(scored, rep_des_s, crit_s, "Bartlett")) |>
  filter(!is.na(level)) |>
  mutate(estimator = factor(estimator, levels = est_levels))

# The design-based domain estimates as one theta function, so the replicate
# covariance between levels exists and the separations can be tested rather than
# read off overlapping intervals. Design-based rather than Bartlett is
# deliberate: the regression scores are shrunk, so a pair resolved here
# separates despite that shrinkage.
dom_meta_cfa <- map(cfg$aux, function(v)
  expand_grid(factor = fnames, level = levels(scored_reg[[v]])) |>
    mutate(variable = v)) |>
  list_rbind() |>
  mutate(segment = match(factor, fnames), idx = row_number())
```

```r
domain_theta_cfa <- function(w_rep) {
  unlist(map(cfg$aux, function(v) {
    M = outer(as.character(scored_reg[[v]]), levels(scored_reg[[v]]), "==") + 0
    M[is.na(M)] = 0
    denom = as.numeric(crossprod(M, w_rep))
    unlist(map(fnames, function(f)
      as.numeric(crossprod(M, w_rep * as.numeric(scored_reg[[f]]))) / denom),
      use.names = FALSE)
  }), use.names = FALSE)
}

est_cfa <- domain_theta_cfa(weights(design_reg$des, "sampling"))
V_cfa_dom <- replicate_variance(rep_des_reg, domain_theta_cfa, est_cfa)

# The same quantity svyby computed, by a different route. Agreement validates
# the ordering the contrasts index into.
chk <- dom_meta_cfa |>
  mutate(p_theta = est_cfa) |>
  inner_join(filter(domain, estimator == "Design-based") |>
               select(variable, level, factor, p),
             by = c("variable", "level", "factor"))
stopifnot(nrow(chk) == nrow(dom_meta_cfa),
          max(abs(chk$p_theta - chk$p)) < 1e-6)

wald_cfa <- list(meta = dom_meta_cfa, est = est_cfa, V = V_cfa_dom,
                 df = degf(rep_des_reg))

wide_gap <- domain |>
  select(variable, level, segment, estimator, p, se) |>
  pivot_wider(names_from = estimator, values_from = c(p, se))

se_ratio <- wide_gap$`se_Design-based` / wide_gap$se_Unweighted

glue("Weighting moves the factor means by a median of ",
     "{round(median(abs(wide_gap$`p_Design-based` -
     wide_gap$p_Unweighted)), 3)}; ",
     "unshrinking them moves a further ",
     "{round(median(abs(wide_gap$p_Bartlett -
     wide_gap$`p_Design-based`)), 3)}. ",
     "Design-based intervals are a median of ",
     "{round(median(se_ratio[is.finite(se_ratio)]), 2)} ",
     "times the naive ones.")
```

```
Weighting moves the factor means by a median of 0.001; unshrinking them moves a further 0.056.
```

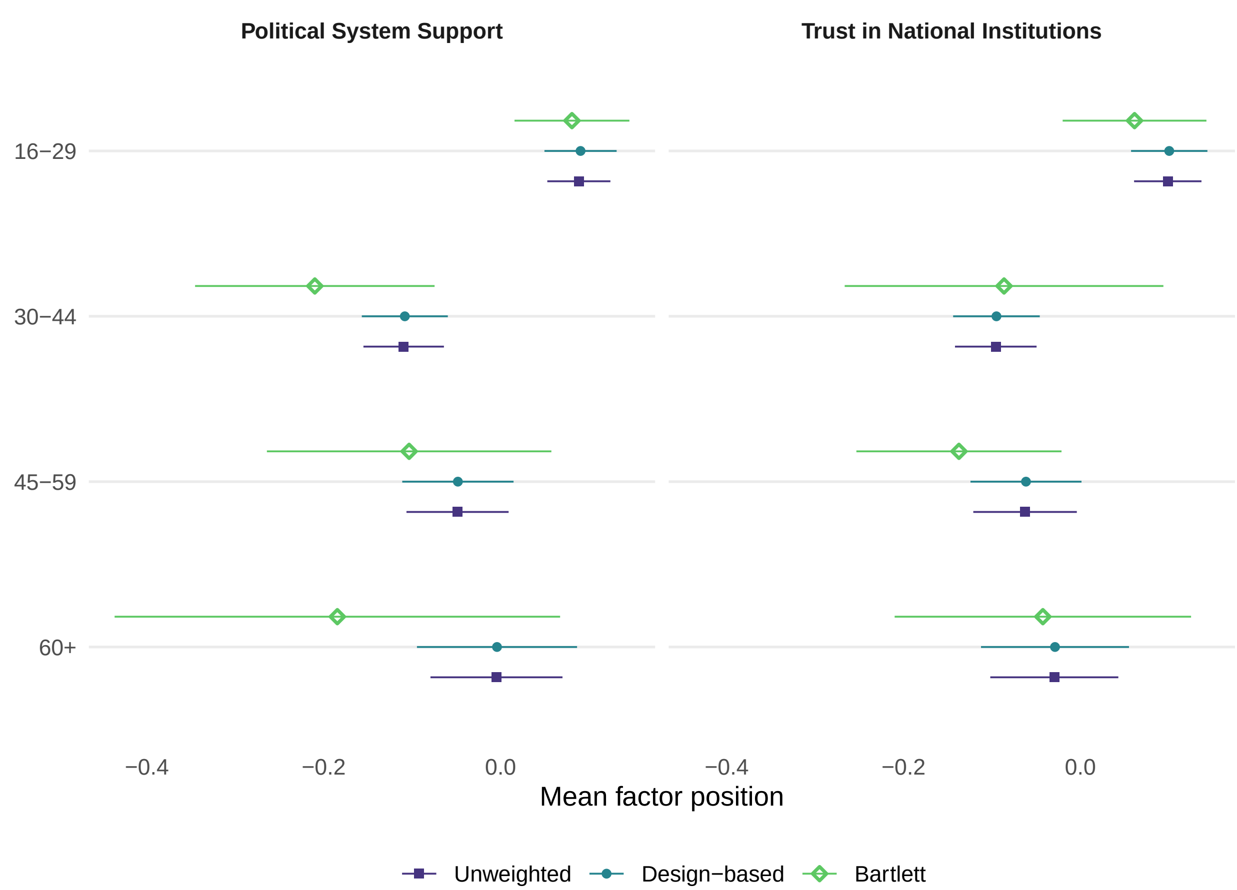
11.1 age_cat

Table 13: Mean factor position by level of age_cat, three ways.

Factor	Estimator	16-29	30-44	45-59	60+
Political System Support	Unweighted	+0.09 (0.018)	-0.11 (0.023)	-0.05 (0.029)	-0.00 (0.038)
Political System Support	Design-based	+0.09 (0.021)	-0.11 (0.024)	-0.05 (0.032)	-0.00 (0.046)
Political System Support	Bartlett	+0.08 (0.033)	-0.21 (0.068)	-0.10 (0.081)	-0.18 (0.127)
Trust in National Institutions	Unweighted	+0.10 (0.019)	-0.10 (0.024)	-0.06 (0.030)	-0.03 (0.037)
Trust in National Institutions	Design-based	+0.10 (0.022)	-0.10 (0.025)	-0.06 (0.032)	-0.03 (0.042)
Trust in National Institutions	Bartlett	+0.06 (0.041)	-0.09 (0.091)	-0.14 (0.058)	-0.04 (0.084)

Table 14: The same positions as an expected answer on each factor's strongest indicator.

Factor	16-29	30-44	45-59	60+
Political System Support	3.61	3.19	3.34	3.23
Trust in National Institutions	3.04	2.83	2.76	2.90



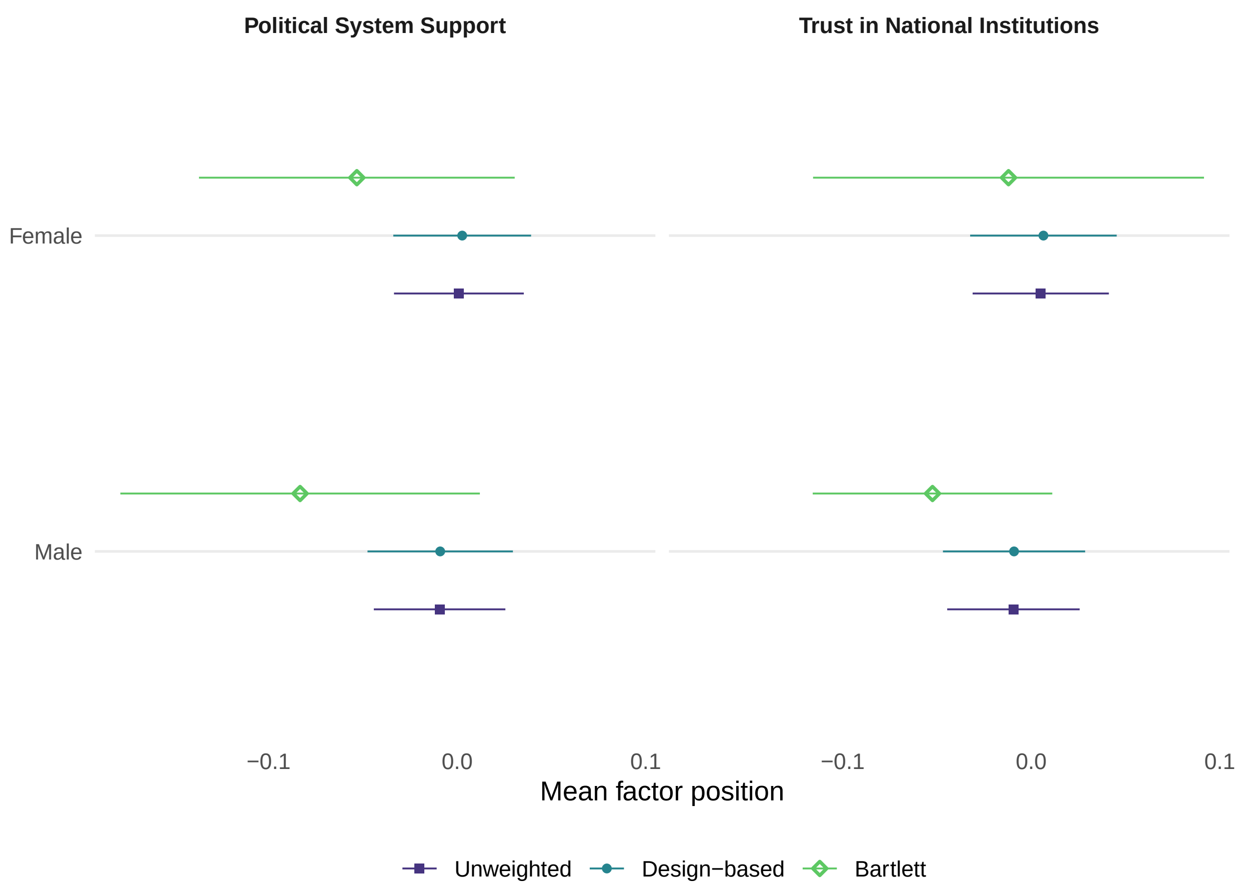
11.2 sex

Table 15: Mean factor position by level of sex, three ways.

Factor	Estimator	Female	Male
Political System Support	Unweighted	+0.00 (0.018)	-0.01 (0.018)
Political System Support	Design-based	+0.00 (0.018)	-0.01 (0.019)
Political System Support	Bartlett	-0.05 (0.042)	-0.08 (0.048)
Trust in National Institutions	Unweighted	+0.01 (0.018)	-0.01 (0.018)
Trust in National Institutions	Design-based	+0.01 (0.020)	-0.01 (0.019)
Trust in National Institutions	Bartlett	-0.01 (0.052)	-0.05 (0.032)

Table 16: The same positions as an expected answer on each factor's strongest indicator.

Factor	Male	Female
Political System Support	3.37	3.42
Trust in National Institutions	2.88	2.94



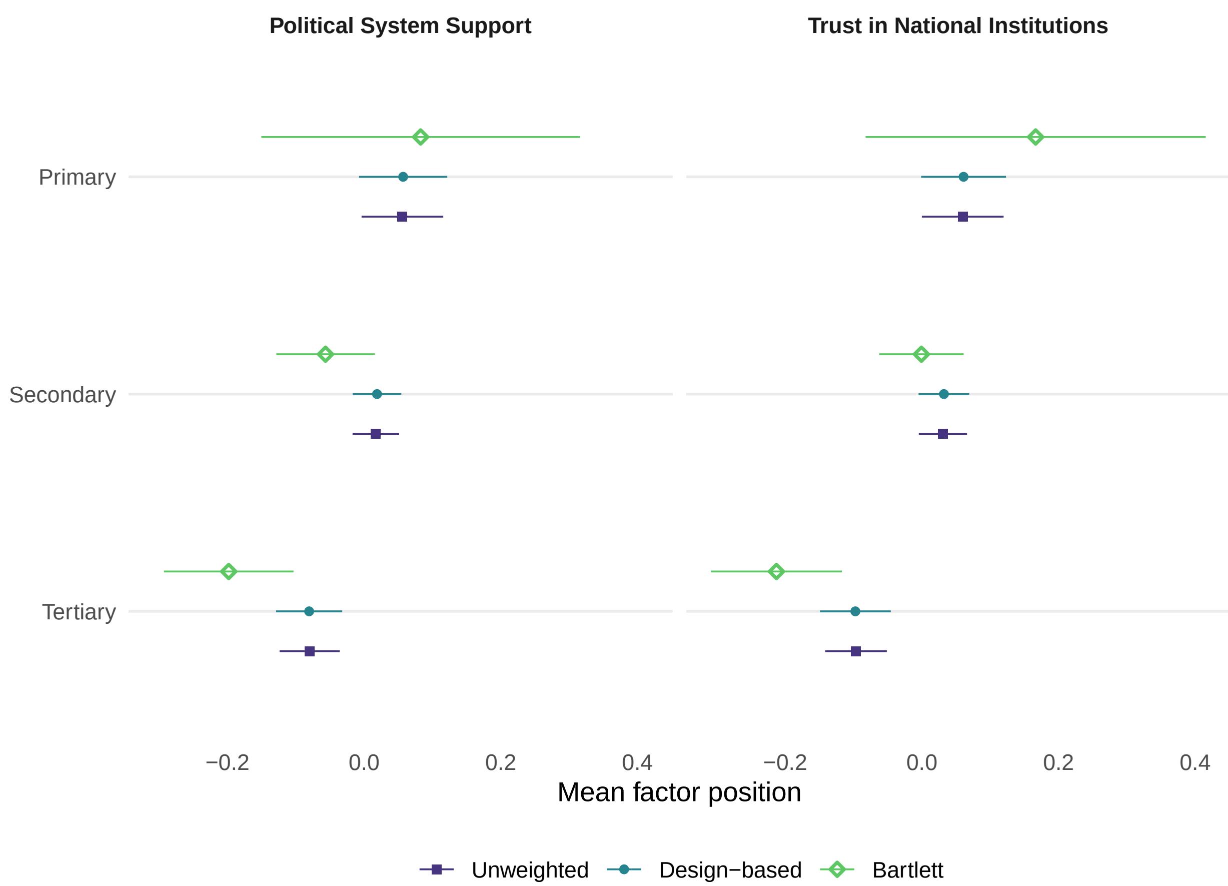
11.3 education

Table 17: Mean factor position by level of education, three ways.

Factor	Estimator	Primary	Secondary	Tertiary
Political System Support	Unweighted	+0.06 (0.031)	+0.02 (0.017)	-0.08 (0.022)
Political System Support	Design-based	+0.06 (0.032)	+0.02 (0.018)	-0.08 (0.024)
Political System Support	Bartlett	+0.08 (0.117)	-0.06 (0.036)	-0.20 (0.048)
Trust in National Institutions	Unweighted	+0.06 (0.031)	+0.03 (0.018)	-0.10 (0.023)
Trust in National Institutions	Design-based	+0.06 (0.031)	+0.03 (0.019)	-0.10 (0.026)
Trust in National Institutions	Bartlett	+0.17 (0.125)	-0.00 (0.031)	-0.21 (0.048)

Table 18: The same positions as an expected answer on each factor's strongest indicator.

Factor	Secondary	Primary	Tertiary
Political System Support	3.41	3.61	3.21
Trust in National Institutions	2.95	3.18	2.66



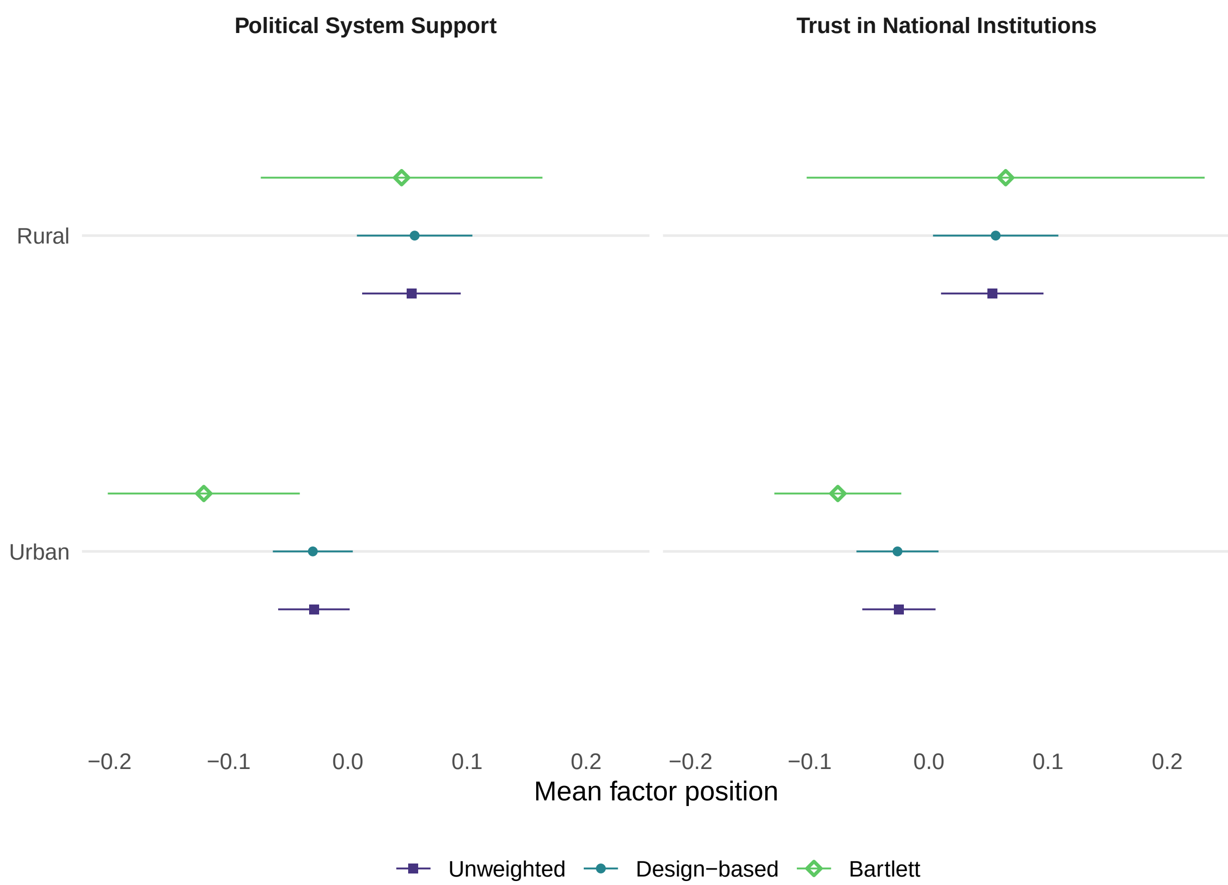
11.4 urban

Table 19: Mean factor position by level of urban, three ways.

Factor	Estimator	Rural	Urban
Political System Support	Unweighted	+0.05 (0.021)	-0.03 (0.015)
Political System Support	Design-based	+0.06 (0.024)	-0.03 (0.017)
Political System Support	Bartlett	+0.05 (0.059)	-0.12 (0.040)
Trust in National Institutions	Unweighted	+0.05 (0.022)	-0.03 (0.016)
Trust in National Institutions	Design-based	+0.06 (0.026)	-0.03 (0.017)
Trust in National Institutions	Bartlett	+0.06 (0.084)	-0.08 (0.027)

Table 20: The same positions as an expected answer on each factor's strongest indicator.

Factor	Urban	Rural
Political System Support	3.32	3.55
Trust in National Institutions	2.85	3.04



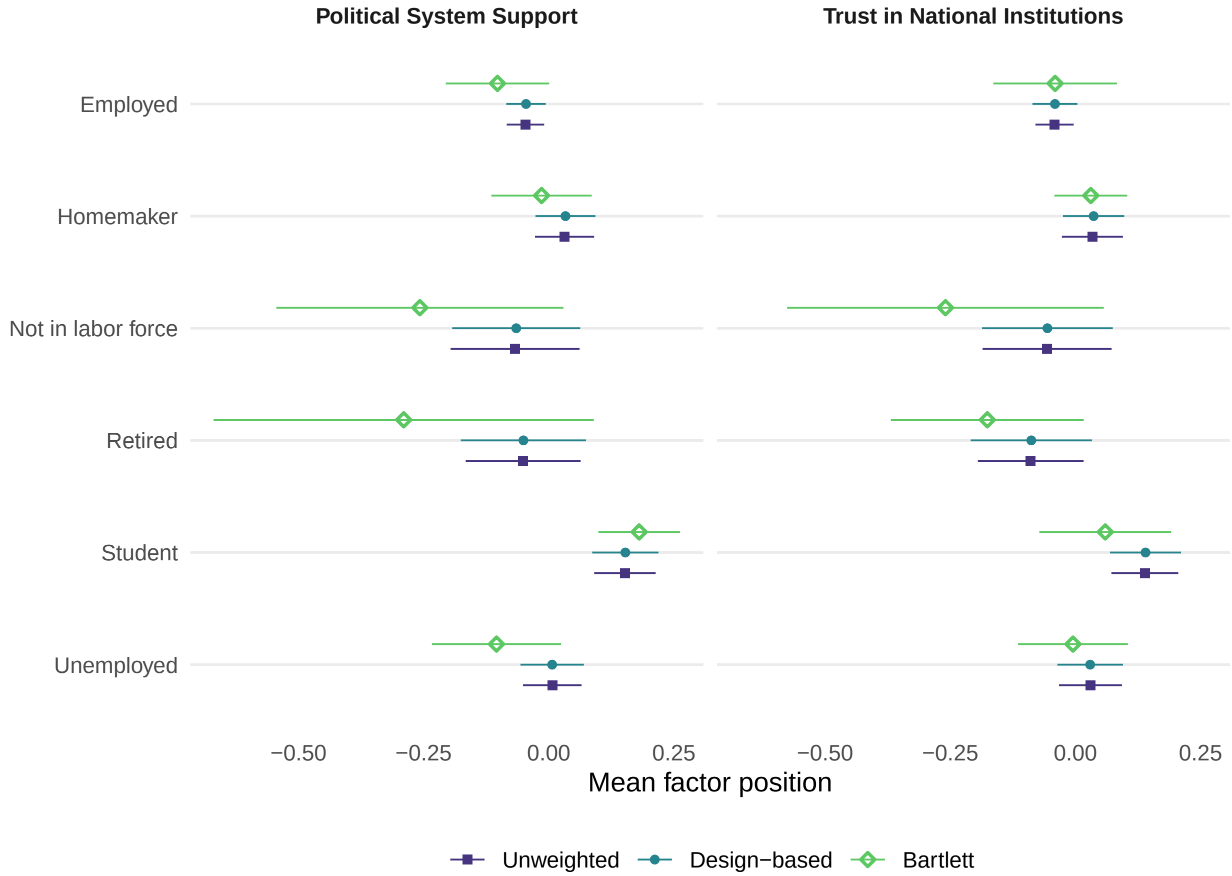
11.5 employment

Table 21: Mean factor position by level of employment, three ways.

Factor	Estimator	Employed	Homemaker	Not in labor force	Retired	Student	Unemployed
Political System Support	Unweighted	-0.05 (0.019)	+0.03 (0.030)	-0.07 (0.066)	-0.05 (0.059)	+0.15 (0.031)	+0.01 (0.030)
Political System Support	Design-based	-0.05 (0.020)	+0.03 (0.030)	-0.07 (0.064)	-0.05 (0.063)	+0.15 (0.033)	+0.01 (0.032)
Political System Support	Bartlett	-0.10 (0.052)	-0.01 (0.050)	-0.26 (0.144)	-0.29 (0.191)	+0.18 (0.041)	-0.10 (0.065)
Trust in National Institutions	Unweighted	-0.04 (0.019)	+0.03 (0.031)	-0.06 (0.066)	-0.09 (0.054)	+0.14 (0.034)	+0.03 (0.032)
Trust in National Institutions	Design-based	-0.04 (0.023)	+0.04 (0.031)	-0.06 (0.066)	-0.09 (0.061)	+0.14 (0.036)	+0.03 (0.033)
Trust in National Institutions	Bartlett	-0.04 (0.062)	+0.03 (0.037)	-0.26 (0.159)	-0.18 (0.097)	+0.06 (0.066)	-0.00 (0.055)

Table 22: The same positions as an expected answer on each factor's strongest indicator.

Factor	Employed	Unemployed	Not in labor force	Student	Homemaker	Retired
Political System Support	3.35	3.34	3.12	3.75	3.47	3.08
Trust in National Institutions	2.90	2.95	2.60	3.04	3.00	2.71



Political System Support
Trust in National Institutions
Employed
Homemaker
Not in labor force
Retired
Student
Unemployed
−0.50
−0.25
0.00
0.25
−0.50
−0.25
0.00
0.25
Mean factor position
Unweighted
Design−based
Bartlett

12 Reading the domain table

The domain table has as many rows as there are demographic levels, and most of them say nothing. The analyst's problem is finding the two or three that do. This section drafts that reading with a language model, and the division of labour is the important part.

The analyst resolves which differences are real; the model only translates. Before anything is sent, R tests every pair of levels: a design-based Wald test on the difference, using the replicate covariance between the two estimates, Holm-adjusted within the demographic. The model is handed the list of pairs that resolve and instructed to name only those. It never sees a standard error and never decides significance. That is the same principle as the naming step: conclusions in, language out.

The tests run on the design-based estimator deliberately. Those scores are shrunk toward the grand mean, so a pair that resolves there separates despite the shrinkage, and the Bartlett columns can only widen the gap.

The rules attached to the prompt forbid ranking levels the analysis did not resolve, forbid causal language, require silence on any level flagged as too small, and cap the output at four findings. The cap is deliberate. The point of this step is to tell an analyst where to look, not to narrate twenty-seven cells.

```
=========== USER PROMPT ( age_cat ) ===========

SURVEY CONTEXT
These items come from the 2023 AmericasBarometer survey of Ecuador, conducted by the
  LAPOP Lab at Vanderbilt University. Fieldwork was carried out face to face in Spanish
  by IPSOS between February and April 2023 with 1,604 respondents, drawn by a
  multi-stage probability design stratified by the three major regions of the country:
  Costa, Sierra, and Oriente.

All items analysed here are answered on the same seven-point ladder anchored at 1 (not
  at all) and 7 (a lot), shown to the respondent on a single card. Four ask how far the
  respondent supports the political system in general; the remaining nine ask how far
  the respondent trusts a specific national institution.
COMPOSITION OF THE POPULATION AND OF EACH GROUP
age_cat in the population: 16-29 42% (n = 625), 30-44 27% (n = 397), 45-59 19% (n =
  285), 60+ 13% (n = 187)
Mean factor position of each level, with 95 percent intervals:
Political System Support
16-29 0.09 [0.05, 0.13], 30-44 -0.11 [-0.16, -0.06], 45-59 -0.05 [-0.11, 0.01], 60+
  -0.00 [-0.09, 0.09]
Trust in National Institutions
16-29 0.10 [0.06, 0.14], 30-44 -0.10 [-0.14, -0.05], 45-59 -0.06 [-0.12, 0.00], 60+
  -0.03 [-0.11, 0.05]

DIFFERENCES THE ANALYSIS RESOLVED
  Criterion: pairwise design-based tests on the replicate covariance, Holm-adjusted
```

within this demographic.
Political System Support: 16-29 differs from 30-44
Political System Support: 16-29 differs from 45-59
Trust in National Institutions: 16-29 differs from 30-44
Trust in National Institutions: 16-29 differs from 45-59
Trust in National Institutions: 16-29 differs from 60+

WHAT THE SURVEY DESIGN CHANGES
Political System Support, 16-29: design moves the estimate by +0.002 and makes the interval 1.14 times as wide
Political System Support, 30-44: design moves the estimate by +0.001 and makes the interval 1.07 times as wide
Political System Support, 45-59: design moves the estimate by +0.000 and makes the interval 1.09 times as wide
Political System Support, 60+: design moves the estimate by +0.001 and makes the interval 1.21 times as wide
Trust in National Institutions, 16-29: design moves the estimate by +0.002 and makes the interval 1.13 times as wide
Trust in National Institutions, 30-44: design moves the estimate by +0.001 and makes the interval 1.06 times as wide
Trust in National Institutions, 45-59: design moves the estimate by +0.001 and makes the interval 1.07 times as wide
Trust in National Institutions, 60+: design moves the estimate by +0.001 and makes the interval 1.16 times as wide

TASK
Write two or three sentences telling an analyst what this variable shows and where to look. Name only the differences listed above. In the caution field, say what accounting for the survey design changed: whether it moved any estimate enough to matter and whether it made the intervals wider or narrower.

RULES:
1. Describe a difference only where it appears in the list above. For any pair not listed, say the data do not separate the groups.
   overlap. Where they overlap, say the data do not separate the groups.
2. Do not rank levels whose intervals overlap.
3. Never use causal language. Groups differ in composition; being in a group does not cause membership.
4. Say nothing about a level flagged as too small.
5. Report at most the four clearest differences. The point is to tell the analyst where to look, not to narrate every cell.
6. When the design changes an estimate or an interval, say so plainly and use the numbers given. Do not treat a narrower interval as better or a wider one as worse; the design-based figure is the honest one either way.
7. Return only valid JSON: no prose before or after, no markdown fences.
8. Each estimate is followed by a 95 percent confidence interval, which expresses sampling uncertainty. Where the analysis did not resolve a pair, say the data do not separate them; do not say they are equal or

```
   similar, since an unresolved pair may differ by more than this sample
   can detect.
JSON (one object): {"finding": "...", "caution": "..."}
```

```r
domain_reads <- map(cfg$aux, function(v) {
  obj <- parse_json_block(
    lca_chat(cfg, persona_cfa)$chat(
      prompt_domain_read(
        domain, marginals, v, labels = factor_labels$Label,
        context = cfg$survey_context,
        values_header =
          "Mean factor position of each level, with 95 percent intervals:",
        wald = wald_cfa),
      echo = FALSE))
  tibble(variable = v,
         Finding = pluck(obj, "finding", .default = NA_character_),
         Caution = pluck(obj, "caution", .default = NA_character_))
}) |>
  list_rbind()

lca_table(domain_reads, widths = c("7em", "22em", "16em"),
          caption = "Drafted reading of each demographic. Verify against the
          tables above.")
```

Table 23: Drafted reading of each demographic. Verify against the tables above.

variable	Finding	Caution
age_cat	Respondents aged 16-29 show higher Political System Support and Trust in National Institutions than those aged 30-44 and 45-59. Regarding Trust in National Institutions, the 16-29 group also differs from those aged 60+, while the data do not separate the other age groups.	Accounting for the survey design moved estimates by at most +0.002 and made the confidence intervals 1.06 to 1.21 times as wide.
sex	The data do not separate men and women on Political System Support or Trust in National Institutions.	Accounting for the survey design moved estimates by no more than +0.002 and made the intervals between 1.06 and 1.11 times as wide.
education	For both Political System Support and Trust in National Institutions, those with tertiary education differ from those with primary and secondary education. The data do not separate the primary and secondary education groups.	Accounting for the survey design moved estimates by no more than 0.002 and made the intervals wider, by a factor between 1.04 and 1.15.
urban	Rural respondents show higher Political System Support (0.06 [-0.01, 0.10]) than urban respondents (-0.03 [-0.06, 0.00]). Rural respondents also show higher Trust in National Institutions (0.06 [0.00, 0.11]) than urban respondents (-0.03 [-0.06, 0.01]).	Accounting for the survey design moved estimates by no more than 0.003 and made the intervals 1.12 to 1.22 times as wide.

Table 23: Drafted reading of each demographic. Verify against *(continued)*

variable	Finding	Caution
employment	Students show higher Political System Support than employed respondents, and higher Trust in National Institutions than both employed and retired respondents. For all other employment pairings, the data do not separate the groups.	Accounting for the survey design moved estimates by no more than 0.002 and made intervals between 0.99 and 1.18 times as wide.

13 The delivered product

```
out_sav <- scored

haven::write_sav(out_sav, file.path(cfg$cfa_dir, "survey_with_factor.sav"))
write_csv(param_tbl, file.path(cfg$cfa_dir, "cfa_measurement_model.csv"))
write_csv(domain, file.path(cfg$cfa_dir, "factor_domains.csv"))
write_csv(marginals, file.path(cfg$cfa_dir, "demographic_marginals_cfa.csv"))

saveRDS(list(fit = fit_w, param_tbl = param_tbl, eig_tbl = eig_tbl,
             efa_tbl = efa_tbl, domain = domain, marginals = marginals,
             items = items, n_factors = cfg$n_factors,
             factor_labels = factor_labels, anchors = anchors),
        file.path(cfg$cfa_dir, "cfa_fit.rds"))

glue("Wrote survey_with_factor.sav, three CSVs, and cfa_fit.rds to",
    cfg$cfa_dir, "\n")
```

```
Wrote survey_with_factor.sav, three CSVs, and cfa_fit.rds toD:/repos/weighted_inference_survey_
```

14 Comparing the two arms

The two reports use different batteries by design, so there is no shared item table to build and no fit contest to run. A `WLSMV` factor model is fitted to a correlation matrix while a latent class model is fitted to the response-pattern likelihood; their information criteria are on different scales and putting them side by side would be wrong.

What the pair demonstrates is the choice itself. This battery is thirteen items of one format asking one kind of question about different objects, and its first eigenvalue dwarfs the second: one continuum, which a factor model represents in a fraction of the parameters a class model would need. The class report's battery is eleven items of mixed format spanning unrelated domains, where a household can own a computer and still have run short of food, and no single continuum holds that. Reading the two search sections against each other is the argument for when each model applies, and it is made by the data rather than asserted.

15 Limitations

Factor scores are estimated rather than observed. Using Bartlett scores removes the shrinkage that would otherwise attenuate the domain estimates in Section 11, which is the factor-analytic counterpart of the bias correction in the class report, but it does so at the cost of wider intervals and it conditions on the fitted measurement model: uncertainty in the loadings and thresholds is not propagated into the domain standard errors.

Under `WLSMV` scoring requires complete items, so respondents who skipped part of the battery are excluded here even though the class report can include them. Where item nonresponse is substantial this is a genuine advantage of the other arm.

Modification indices ignore the clustering and are inflated by roughly the design effect on the loadings. They are reported as a ranking and no threshold is quoted.

The structure search and the confirmatory fit use the same data. Where `cfg$cfa_factors` was set from this report's own exploratory results, the confirmatory fit is a re-expression of that structure rather than an independent test of it, and its fit indices are optimistic. The replicate intervals on the eigenvalues in Section 5 are the guard against reading noise as structure, and they are not a substitute for an independent sample. The two-factor solution fitted here departs from the single factor the eigenvalues support, on the substantive grounds stated in Section 6.

The replicate count equals the number of primary sampling units, so joint tests across more parameters than there are replicates are unavailable. Individual standard errors and pairwise covariances are unaffected, which is all the pairwise contrasts in Section 12 require.

Names drafted by a language model are drafts with an audited override path. No reliability estimate across runs is reported, and every substantive claim rests on the estimated loadings rather than on the name.

References

Rosseel, Yves. 2012. "Lavaan: An R Package for Structural Equation Modeling." *Journal of Statistical Software* 48 (2): 1–36. https://doi.org/10.18637/jss.v048.i02.